

Verzija: promotivno-standardizacioni nacrt -- Version: Promotional-standardization draft*

GO-1 do GO-5

Predlog za standardizaciju, popularizaciju i tehničku evaluaciju lokalnih LLM/LMM benchmark zadataka

Proposal for standardization, popularization and technical evaluation of local LLM/LMM benchmark tasks

dr Jovan Ivković¹

Metodološki esej / naučni rad - Methodological essay / scientific paper

1. Email: jovan.ivkovic@its.edu.rs, ORCID: <https://orcid.org/0000-0001-5236-9529>

Ovaj dokument formalizuje benchmark skup GO-1 do GO-5 kao mali, ali metodološki gust okvir za procenu lokalnih i edge LLM/LMM sistema u režimu stroge formalne tačnosti, višejezičkog praćenja instrukcija, strukturisanog izlaza, determinističkog generisanja instanci i praktične izvršivosti koda.

Dokumentuje se poreklo benchmarka, dizajnerska filozofija, standardizovana struktura odgovora, pravila bodovanja, tipične greške, predlog protokola za javnu upotrebu, kao i aneks sa kanonskim formulacijama zadataka od GO-1 do GO-5.

Sažetak

GO-1 do GO-5 predstavljaju namenski projektovan benchmark skup za procenu lokalnih i edge LLM/LMM sistema u uslovima u kojima generički leaderboard-i i arene nisu dovoljni. Ovi zadaci nisu nastali kao demonstracioni problemi, već kao odgovor na praktičnu potrebu da se kvantizovani modeli, pokrenuti na ograničenom hardveru i pod različitim inference podešavanjima, uporede prema realnoj upotrebljivosti: strogoj tačnosti, disciplini formata, kvalitetu međukoraka, kvalitetu koda, determinističkom generisanju instance i transparentnosti mašinski proverljivog izlaza. Ovaj rad definiše poreklo benchmarka, njegovu idejnu osnovu, opseg svake pojedinačne probe, sistem kvantifikacije ocena, kaznenu logiku za tipične greške, protokol izvođenja i preporuke za popularizaciju i standardizovanu primenu. U radu se pokazuje da mali broj pažljivo konstruisanih zadataka može razotkriti suštinske razlike među modelima mnogo direktnije od širokih, ali manje fokusiranih benchmark zbirki. GO set je osmišljen da meri ono što je u praksi presudno: da li je model ne samo ubedljiv, nego i proverljivo tačan, strukturisan, generativno disciplinovan i tehnički upotrebljiv.

Ključne reči: lokalni LLM, edge AI, benchmark metodologija, višejezička evaluacija, formalna tačnost, structured output, bitmask reasoning, code-grounded evaluation.

Executive summary

- GO benchmark set je namenjen situacijama u kojima korisnika ne zanima samo prosečan leaderboard skor, već realna upotrebljivost modela u lokalnom radu.
- Pet zadataka zajedno pokrivaju formalnu logiku, egzaktno numeričko rezonovanje, prostorno-algebarski ručni račun, kombinatoriku i pretragu u Zig-u, kao i determinističko generisanje formalne instance, forenzičku rekonstrukciju i proveru jedinstvenosti u savremenom Zig 0.16.x+ okruženju.
- Benchmark je namerno višejezički i strogo strukturisan. Srpski jezik, precizna sintaksa izlaza i obavezne A/B/C/JSON sekcije predstavljaju svesno dizajnersko opterećenje testa.
- Standardizacija benchmarka zahteva ne samo fiksne tekstove zadataka, nego i jasan protokol bodovanja, čuvanje svih rerunova, transparentan izbor reprezentativnog najboljeg rezultata po modelu i mogućnost audit-varijanti koje menjaju instancu bez promene metodologije.

1. Uvod: zašto je ovaj benchmark bio potreban

Dominantni javni benchmark ekosistem odlično meri opšte sposobnosti modela nad velikim korpusima zadataka, ali relativno slabo meri ono što se u realnom lokalnom radu pokazuje presudnim: formalnu tačnost pomoćnih brojki, disciplinu odgovora, kvalitet mašinski proverljivog izlaza, stabilnost pod kvantizacijom i sposobnost da kod bude blizak stvarnoj izvršivosti.

U lokalnim i edge scenarijima model nije abstraktna inteligencija nego radni alat. Pokreće se pod ograničenim RAM/VRAM budžetom, uz CPU/GPU offload, promenljive prompt template-ove, različite quantizacije i često bez spoljne pomoći. U takvim uslovima model koji je prosečno dobar, ali formalno nedisciplinovan, može biti manje koristan od modela koji je skromniji, ali proverljivo tačan.

GO-1 do GO-5 nastali su kao odgovor na upravo taj problem. Cilj nije bio da se pronade najreklamiraniji model, već da se uvede mali skup zadataka koji će meriti ono što korisnik tehničkog modela zaista želi: da li model razume, računa, prati format, daje ispravan JSON, i proizvodi kod koji je bliži alatu nego ukrasu.

2. Idejno poreklo benchmarka

Benchmark skup GO-1 do GO-5 nastao je iz iterativnog procesa lokalnog testiranja otvorenih i kvantizovanih modela različitih veličina, arhitektura i inferencijskih režima. Tokom tog procesa postalo je očigledno da standardni javni benchmark signali nisu dovoljni da objasne zašto se model odlično ponaša u jednoj tabeli, a razočaravajuće u realnom tehničkom radu.

Osnovna ideja pri konstrukciji seta bila je trostruka. Prvo, svaki zadatak mora imati jasno definisan tačan ishod i dovoljno malu dimenziju da ljudski evaluator može ručno proveriti rezultat. Drugo, svaki zadatak mora terati model da otkrije više od jedne sposobnosti odjednom: logiku, račun, disciplinu, kod i JSON. Treće, skup zadataka mora biti dovoljno raznolik da model ne može proći samo zato što je jak u jednom tipu problema.

Zbog toga GO set nije ni čisto matematički, ni čisto koderski, ni čisto jezički benchmark. To je slojeviti skup koji kombinuje formalnu logiku, diskretnu dinamiku, linearnu algebru, sistemsko programiranje i strogi izlazni ugovor. Upravo ta kombinacija čini ga pogodnim za evaluaciju suverenih lokalnih modela.

3. Šta benchmark treba da pokaže i čemu treba da pomogne

Primarna svrha benchmarka nije akademska dekoracija, već praktična diferencijacija modela. GO set, uključujući novi GO-5, treba da pomogne istraživaču, inženjeru ili naprednom korisniku da odgovori na pitanja:

koji model stvarno radi dobro pod kvantizacijom; koji model bolje poštuje strukturisani output; koji model ima stvarni koderski potencijal; koji model je najbolji kao referenca, a koji kao workhorse ili brzi filter.

Druga svrha benchmarka je standardizacija internog i eksternog izveštavanja. Ako više testera koristi isti skup, iste sekcije odgovora, ista pravila bodovanja i isti način prijave rezultata, tada poređenja postaju transparentnija. To otvara prostor da se GO set koristi i u promotivne svrhe, i u ozbiljnim tehničkim evaluacijama.

Treća svrha benchmarka je edukativna. Dobro konstruisan benchmark nije samo mera, već i objašnjenje. Kroz GO zadatke postaje jasno zašto su formalni detalji važni, zašto JSON nije ukras, zašto jezik zadatka menja ponašanje modela i zašto mala sintaksna greška u kodu može oboriti inače lep odgovor.

4. Osnovni arhitektonski principi GO seta

Princip	Obrazloženje
Mala dimenzija, velika proverljivost	Svaki zadatak je dovoljno mali da ga evaluator može ručno proveriti, ali dovoljno gust da razotkrije duboke greške.
Višeslojna evaluacija	Ne ocenjuje se samo krajnje rešenje, već i međukoraci, formalna struktura, kod i JSON.
Višejezičko opterećenje	Srpski jezik uvodi dodatno test opterećenje i smanjuje oslanjanje na memorisane engleske benchmark obrasce.
Praktični kod	Traženi programski jezici i stilovi nisu slučajni: Python, C i Zig namerno proveravaju različite nivoe formalne kontrole.
Transparentna kaznena logika	Greške u pomoćnim brojkama, JSON-u ili sintaksi ne smeju biti sakrivene iza lepog narativa.
Kompatibilnost sa lokalnim radom	Benchmark mora biti relevantan za kvantizovane modele, hibridni CPU/GPU rad i edge okruženja.

Tabela 1. Temeljni dizajnerski principi benchmark skupa.

5. Zašto su jezik, sintaksa i struktura odgovora deo testa

GO set je namerno formulisan na srpskom jeziku. To nije estetski dodatak, već metodološka odluka. Modeli su dominantno trenirani, fino podešavani i benchmarkovani na engleskim zadacima, pa upotreba srpskog smanjuje verovatnoću da model samo prepozna već viđen obrazac bez stvarnog razumevanja.

Obavezne sekcije tipa A, B, C i RESULTS_JSON uvedene su da bi se razdvojile logika, analiza, kod i mašinski proverljiv izlaz. Ovakva struktura sprečava model da sakrije grešku iza dugačkog stila pisanja. Ako je model pogrešio, evaluator odmah vidi da li je problem nastao u logici, numerici, implementaciji ili u završnom JSON objektu.

Traženi programski jezici nisu izabrani nasumično. Python proverava egzaktno računanje i jednostavnu proveru. C proverava razumevanje memorijskog modela, indeksiranja i proceduralne implementacije. Zig proverava strogu sintaksu, tipove, kompatibilnost sa aktuelnim standardnim bibliotečkim modelom i sposobnost da se napravi mali, ali realan solver ili verifikator. Zbog toga sintaksa jezika odgovora nije sporedna stvar, nego centralni deo benchmark filozofije.

6. Detaljan opis benchmark zadataka

6.1 GO-1 — logička korekcija matrice dozvola uz minimalni broj izmena

GO-1 je test formalno-logičkog rezonovanja nad ograničenjima. Model dobija matricu uloga i dozvola i skup pravila koja međusobno povezuju bitove dozvola. Zadatak nije samo da detektuje kršenja, nego da pronađe minimalan skup bit-flip izmena koji sva ograničenja dovodi u zadovoljen status, da prikaže finalnu matricu, i da obrazloži zašto je rešenje minimalno i jedinstveno.

Ovaj test razotkriva modele koji umeju da prepoznaju lokalna kršenja, ali ne umeju da razmišljaju globalno. U praksi se često dešava da model popravi jedno ograničenje, a time naruši drugo, ili da pronađe važeće rešenje sa više izmena nego što je potrebno. GO-1 zato ne meri samo logiku, nego i optimizacionu disciplinu.

Tražena struktura odgovora je: A-KRSENJA, B-KOREKCIJA, C-DOKAZ_MINIMALNOSTI i RESULTS_JSON. Ovaj raspored tera model da razdvoji dijagnostiku od korekcije i korekciju od dokaza.

GO-1 element ocene	Poeni / 10
Tačna identifikacija svih prekršenih ograničenja	2.0
Tačan minimalni broj bit-flip promena	2.0
Tačan skup promena i ispravna finalna matrica	2.5
Dokaz minimalnosti	2.0
Tvrdnja o jedinstvenosti i njeno obrazloženje	1.0
JSON disciplina	0.5

Tabela 2. GO-1 rubrika i raspodela ocene.

6.2 GO-2 — LIF neuron, egzaktna aritmetika i dokaz invarijante

GO-2 proverava sposobnost modela da istovremeno drži pod kontrolom diskretnu dinamiku sistema, formalni dokaz, tačan tablični prikaz i proverljiv Python kod. Model dobija jednačine LIF neurona, ulazni niz i početni uslov. Potrebno je dokazati invarijantu $0 \leq v_t < 1$, izračunati sve korake i isporučiti rezultujući JSON objekat.

Posebna vrednost GO-2 je u tome što kažnjava modele koji 'razumeju priču', ali kvare brojeve. Veliki broj modela može dati ubedljivo objašnjenje dinamike i pogrešiti tek poslednje decimale ili jedan pragovni slučaj. U tehničkoj upotrebi upravo su takve greške najopasnije, jer se lako previdi da je narativ lep, a broj netačan.

Struktura odgovora A-IZVOD, B-TABELA, C-KOD i RESULTS_JSON namerno tera model da održi konzistentnost između objašnjenja, ručnog računa i programskog izlaza.

GO-2 element ocene	Poeni / 10
Korektan dokaz invarijante	2.5
Tačni razlomci po koracima	2.5
Tačne decimale po koracima	1.5

Tačna tabela i broj spike-ova	1.5
Python kod koji reprodukuje rezultat	1.5
JSON disciplina	0.5

Tabela 3. GO-2 rubrika i raspodela ocene.

6.3 GO-3 — validna 2D konvolucija, ručni rezultat i C implementacija

GO-3 kombinuje ručni račun, linearnu algebru, prostornu intuiciju i sistemsko programiranje. Model mora da rotira kernel za 180 stepeni, ručno izračuna svaki izlazni element, objasni izlazne dimenzije i broj operacija, i potom napiše C implementaciju validne 2D konvolucije.

Ovaj test je veoma efikasan u razotkrivanju modela koji koncept konvolucije poznaju površno. Tipične greške uključuju mešanje konvolucije i korelacije, pogrešno rotiranje kernela, netačno sabiranje međuproizvoda, loše row-major objašnjenje ili C kod koji izgleda profesionalno, ali je zapravo nepouzdan.

GO-3 je posebno vredan jer spaja ručni i koderski deo. Model ne može da prođe samo na osnovu jednog od ta dva sloja.

GO-3 element ocene	Poeni / 10
Tačan ručni izlaz	3.0
Tačni međukoraci	1.5
Tačna analiza dimenzija i broja operacija	2.0
Objašnjenje row-major indeksiranja	1.0
Kvalitet C koda	2.0
JSON disciplina	0.5

Tabela 4. GO-3 rubrika i raspodela ocene.

6.4 GO-4 — Latin-square solver, bitmaske, Zig i strogi output contract

GO-4 je kulminacija benchmark seta. Zadatak od modela traži da analizira mali 4x4 Latin-square problem, izbroji prazna polja i početne singletoni, izvede najmanje četiri prisilna koraka, objasni bitmask pristup i napiše mali Zig rešavač sa proverom jedinstvenosti.

GO-4 je namerno najteži jer spaja četiri osetljive tačke: pomoćne brojke, formalnu logiku, striktan sintaktički jezik i mašinski proverljiv JSON. Model može imati tačno konačno rešenje, a ipak dobiti osetan odbitak poena ako su početni singletoni pogrešno izbrojani, ako je Zig samo pseudokod ili ako je JSON u nesaglasnosti sa tekstom.

U praksi se upravo GO-4 pokazuje kao najjači filter za modele koji zvuče ubedljivo, ali nemaju dovoljnu tehničku disciplinu. Zato se u kompozitnoj interpretaciji često smatra najvrednijim pojedinačnim testom.

GO-4 element ocene	Poeni / 10
Tačan broj praznih polja i početnih singletona	1.5

Tačni skupovi kandidata i bitmaske	1.5
Najmanje četiri korektna prisilna koraka	1.5
Dokaz / potvrda jedinstvenosti	1.0
Objašnjenje bitmask pristupa	1.0
Zig kod — algoritamska ispravnost	2.0
Zig kod — sintaksa i struktura	1.0
JSON disciplina	0.5

Tabela 5. GO-4 rubrika i raspodela ocene.

6.5 GO-5 — seedovani Mealy transduktor, forenzička korekcija i Zig 0.16.x+ verifikator

GO-5 je projektovan kao dodatni integrisani test koji spaja determinističko generisanje formalne instance, ručnu simulaciju formalnog sistema, forenzičku identifikaciju jedne greške, dokaz jedinstvenosti i savremeni Zig 0.16.x+ kod. Za razliku od GO-1 do GO-4, gde model dobija već formulisanu instancu problema, ovde mora prvo da je konstruiše iz zadatog seed-vektora, pa tek potom da je ručno i algoritamski obradi. Time se meri nova klasa discipline: sposobnost da model ne samo reši problem, nego i da tačno reprodukuje njegovu unutrašnju generativnu logiku.

Metodološki razlog za uvođenje GO-5 je jednostavan. U praksi se pokazuje da veliki broj modela može da deluje veoma ubedljivo kada dobije gotovu tabelu, gotovu matricu ili gotove ulazne veličine, ali gubi formalnu pouzdanost kada mora da održi pun lanac konzistentnosti: generator pravila → parametri sistema → kandidatske instance → ručna simulacija → dokaz jedinstvenosti → izvršiv kod → mašinski proverljiv JSON. GO-5 je namerno napravljen tako da jedna rana pomoćna greška kontaminira sve kasnije sekcije, čime se razotkriva sklonost modela ka samouverenom nastavljanju nad pogrešnim međurezultatima.

Formalni domen GO-5 je mali deterministički Mealy transduktor sa tri kandidatske ulazne reči, jednim posmatranim izlazom i pretpostavkom tačno jedne simbolske greške. Ovaj izbor je metodološki povoljan zato što je zadatak dovoljno mali za ručnu proveru, a dovoljno gust da istovremeno testira modularnu aritmetiku, state tracking, Hamming-ova rastojanja, kontrafaktualnu proveru i aktuelnu Zig sintaksu. Time GO-5 zauzima mesto između čiste formalne logike i sistemskog programiranja, ali dodaje i jedan novi sloj: model mora tačno da generiše problem koji zatim rešava.

GO-5 je takođe osmišljen kao eksplicitan test protiv Goodhart-ove degradacije benchmarka. Ako javna instanca vremenom postane poznata, evaluacioni signal se ne čuva dodatnim narativnim komplikovanjem, već audit-varijantama istog formalnog generatora. Drugim rečima, isti benchmark domen može davati više instanci nad različitim seed-vrednostima, bez promene rubrike i bez narušavanja uporedivosti. Kanonska javna instanca zato služi kao referenca, a audit-varijante kao zaštita od mehaničkog memorisanja odgovora.

Sekcije GO-5 odgovora imaju strogo podeljene funkcije. A-GENERISANJE meri da li model tačno prati pravila konstrukcije sistema. B-RUCNA_SIMULACIJA meri disciplinu održavanja stanja kroz više kandidata i više koraka. C-DOKAZ_JEDINSTVENOSTI proverava da li model ume da razlikuje pronalazak jednog dobrog objašnjenja od stvarnog isključenja svih alternativa. D-ZIG_KOD proverava da li formalno rezonovanje može da bude pretočeno u izvršiv, savremeno kompatibilan Zig 0.16.x+ program. RESULTS_JSON je završni mašinski proverljiv sloj koji ne sme biti u kontradikciji sa ručnim ili koderskim delom.

Za lakši ljudski pregled i strože ocenjivanje, ručni deo GO-5 je namerno preveden u obavezne mini-tabele. Model mora da prikaže kompletnu tabelu transduktora, tabelu generisanja kandidatskih reči, tabelu ručne simulacije po kandidatima i zbirnu tabelu konačnih izlaza, putanja i rastojanja. Ovakav format smanjuje prostor da model zamagli grešku narativom i ujedno olakšava forenzičku proveru da li su `candidate_outputs` i `candidate_distances` u JSON-u zaista izvedeni iz ručne simulacije, a ne samo nasumično pogodni za finalni zaključak.

GO-5 element ocene	Poeni / 10
Tačno generisanje parametara, tabele transduktora i reči	2.0
Tačna ručna simulacija sva tri kandidata po koracima	2.0
Tačna zbirna tabela izlaza, putanja i Hamming-ovih rastojanja	0.5
Dokaz jedinstvenosti i inverzna provera	2.0
Zig kod — generisanje i simulacija bez hardkodovanja	1.2
Zig kod — Zig 0.16.x+ sintaksa, I/O i kompatibilnost	1.0
Zig kod — pretraga svih validnih objašnjenja i <code>solveUnique</code> logika	0.8
JSON disciplina i saglasnost sa B/C sekcijama	0.5

Tabela 6. GO-5 rubrika i raspodela ocene.

7. Horizontalna pravila evaluacije i kaznene logike

Benchmark je dizajniran tako da nije dovoljan samo tačan konačni odgovor. Pomoćne brojke, međukoraci, sintaksa i JSON tretiraju se kao integralni delovi rešenja. Time se sprečava da model pogodi finalnu matricu ili broj, a da pri tome pokaže formalno nepouzdan tok rada.

Greške se razvrstavaju u tri klase. Kritične greške su one koje ruše suštinu rešenja: pogrešan minimalni broj promena, pogrešan ručni izlaz, pogrešan broj spike-ova, pogrešno konačno rešenje ili suštinski pogrešan solver. Srednje greške su npr. tačan razlomak ali pogrešna decimala, loš dokaz minimalnosti ili sintaktički problem koji je lako ispravljiv. Male greške su tipografske, formatne i stilske, ako ne narušavaju formalnu tačnost.

Posebno pravilo važi za reasoning blokove. Ako je thinking deo tehnički odvojen od finalnog odgovora u korisničkom interfejsu, sklopiv i ne kontaminira traženi A/B/C/JSON izlaz, ne tretira se kao curenje u finalni odgovor. Penalizacija se uvodi samo kada reasoning uđe u isti izlazni blok i naruši output contract.

Kod GO-5 dodatno se izdvajaju specifične klase grešaka koje se u drugim testovima ređe vide. Prva je generator drift: model pogreši u parametru transduktora ili u jednoj komponenti generisane reči, pa zatim dosledno rešava pogrešan problem. Druga je simulacioni drift: jedan kandidat bude ispravno praćen, ali drugi ili treći izgubi stanje ili izlaz posle nekoliko koraka. Treća je correct-by-coincidence situacija, kada model slučajno pogodi pobednički kandidat, ali su `candidate_outputs` ili `candidate_distances` netačni. Četvrta je poslednji-kilometar greška u Zig 0.16.x+ sloju, najčešće kroz zastareli pre-0.16 I/O obrazac ili sintaksu koja deluje ubedljivo, ali nije kompatibilna sa aktuelnim standardnim bibliotečkim modelom.

U evaluaciji GO-5 važi dodatno normativno pravilo: sekcija C ne može dobiti punu ocenu ako sekcija B nije tačna za sva tri kandidata. Takođe, `RESULTS_JSON` se smatra suštinski pogrešnim ako `candidate_outputs` ili `candidate_distances` nisu strogo saglasni sa ručnom simulacijom, čak i kada je finalni `matching_word_index`

slučajno tačan. Ovim se sprečava da model prođe samo na osnovu lepog završnog zaključka bez valjanog srednjeg lanca.

Klasa greške	Tipični primeri	Efekat na ocenu
Kritična greška	Pogrešno konačno rešenje, pogrešan minimalni broj promena, pogrešan JSON ključni sadržaj, bitno pogrešan solver	Veliki odbitak; test često pada ispod 5/10
Srednja greška	Pogrešne pomoćne brojke, nedovoljno ubedljiv dokaz, sintaksno slab ali prepoznatljiv kod	Umereni odbitak; test ostaje prolazan ali ne vrhunski
Mala greška	Stilske nedoslednosti, sitna formatna odstupanja, manje nepreciznosti bez posledica po suštinu	Mali odbitak

Tabela 7. Horizontalna kaznena logika benchmark seta.

8. Predlog standardnog protokola izvođenja

Da bi benchmark bio uporediv, potrebno je standardizovati ne samo tekst zadatka već i način izvođenja. Minimalni skup metapodataka koji se uz rezultat prijavljuje obuhvata naziv modela, kvantizaciju, prompt template, sampling podešavanja, kontekst, broj generisanih tokena, TTFT, tokene po sekundi, ukupno vreme i po mogućnosti RAM/VRAM opterećenje.

Preporučuje se najmanje jedan primarni run po testu i po potrebi više rerunova ako model pokazuje osetljivost na template ili sampling. Svi rerunovi treba da budu sačuvani, ali se kao reprezentativni rezultat po modelu i testu preporučuje najbolji kvalitet postignut pod stabilnim i dokumentovanim uslovima. Takva praksa je u skladu sa eksperimentalnim radom u kojem se ne prijavljuje samo prvi slučajni prolaz, već reproducibilan maksimum modela pod jasno opisanim režimom.

Za javnu popularizaciju preporučuje se da se benchmark objavljuje zajedno sa kanonskim promptovima, JSON šemama, skalom ocenjivanja i primerom jednog idealnog referentnog rešenja po testu. Time se smanjuje proizvoljnost i jača poverenje zajednice.

- Koristiti identičan tekst zadatka za poređenje modela iste generacije.
- Zapisati sve inference parametre i prompt/template režim.
- Čuvati sirove logove i reprezentativne finalne odgovore.
- Za GO-4 prijaviti i sve pomoćne brojke, ne samo konačnu matricu.
- Razdvojiti 'all runs' tabelu od 'best by model' tabele.
- Na grafikonima jasno naznačiti da li se prikazuju svi runovi ili samo najbolja tačka po modelu.
- Za GO-5 čuvati i dokaz kompilacije i runtime izlaza za Zig 0.16.x+ kod, ne samo tekst odgovora.
- Razdvojiti javnu kanonsku instancu GO-5 od audit-varijanti sa drugim seed-vrednostima i drugim posmatranim izlazima, uz istu rubriku.
- Za GO-5 beležiti da li je model pogrešio u generatoru, ručnoj simulaciji, dokazu jedinstvenosti, Zig kompatibilnosti ili JSON saglasnosti; ta klasifikacija je podjednako važna kao zbirna ocena.

9. Predlog za popularizaciju benchmarka

Za uspešnu popularizaciju benchmark mora biti predstavljen ne samo kao skup zadataka nego kao jasan odgovor na realan problem zajednice: kako razlikovati lokalne modele koji su samo retorički jaki od onih koji su stvarno tehnički pouzdani. Zbog toga je u promotivnim materijalima važno isticati da GO set meri upotrebljivost, a ne samo opštu pamet modela.

Drugo, benchmark treba promovisati kroz transparentnost. Javno dostupni promptovi, opis rubrika, primeri dobrih i loših odgovora, kao i jasan protokol prijave rezultata, učiniće set kredibilnim i lakim za usvajanje. Treće, benchmark treba prezentovati i kao obrazovni alat: kroz njega korisnici uče zašto su JSON, pomoćne brojke, sintaksa i međukoraci jednako važni kao i krajnja tačnost.

Za širu zajednicu preporučljivo je koristiti dve paralelne reprezentacije. Prva je tehnički rad i standardizovana dokumentacija. Druga je sažeti promotivni 'benchmark brief' od nekoliko stranica sa grafikonima, FAQ sekcijom i primerima tipičnih modelskih grešaka.

10. Zaključak

GO-1 do GO-5 čine mali, ali metodološki bogat benchmark skup namenjen lokalnim i edge LLM/LMM scenarijima. Njegova glavna snaga je što ne meri samo inteligenciju modela u apstrakciji, već njegovu tehničku upotrebljivost: da li model prati format, računa tačno, objašnjava konzistentno, daje proverljiv JSON, generiše instancu iz pravila i proizvodi kod koji je blizak realnom alatu.

Ovaj benchmark treba posmatrati kao dopunu velikim javnim leaderboard-ima, a ne kao konkurenciju njihovoj ulozi. Njegov doprinos je u tome što uvodi veoma osetljivu i transparentnu procenu formalne discipline modela — osobine koja je od presudne važnosti u ozbiljnom lokalnom radu.

Najvažnija poruka ovog skupa glasi: nije dovoljno da model bude ubedljiv; mora biti proverljivo tačan, strukturisan i tehnički disciplinovan. Upravo zbog toga GO set ima potencijal da postane korisna referenca za buduću standardizaciju evaluacije lokalnih modela.

Aneks A. Kanonske postavke benchmark zadatka

Ovaj aneks daje standardizovane, skraćene i promotivno-upotrebljive formulacije samih benchmark zadataka od GO-1 do GO-5. Tekst može služiti kao javna dokumentacija, kao osnova za prenos benchmarka u druge sisteme i kao referentni opis za reprodukciju.

A.1 GO-1 — Kanonska formulacija

Zadat je skup uloga {CEO, CFO, OPS, AUD} i binarna matrica dozvola nad kolonama {D, A10, A50, VC, VP}. Zadati su sledeća ograničenja:

- 1) CEO mora imati $D=1$.
- 2) Ako neka uloga ima $A50=1$, mora imati i $A10=1$.
- 3) Tačno jedna od uloga {CEO, CFO} mora imati $A50=1$.
- 4) OPS mora imati $D=1$ i $A10=1$.
- 5) Nijedna uloga ne sme istovremeno imati $VC=1$ i $VP=1$.
- 6) Tačno jedna od uloga {CFO, OPS} mora imati $VC=1$.
- 7) CFO mora imati $VP=1$.
- 8) AUD sme imati samo $D=1$, a sve ostale dozvole 0.

Traži se:

A-KRSENJA: navesti prekršena ograničenja;

B-KOREKCIJA: pronaći minimalan broj bit-flip promena i finalnu matricu;

C-DOKAZ_MINIMALNOSTI: dokazati minimalnost i obrazložiti jedinstvenost;

RESULTS_JSON: isporučiti mašinski proverljiv JSON sa poljima `violated_constraints`, `bit_flips_count`, `bit_flips`, `final_matrix` i `is_unique_minimum`.

A.2 GO-2 — Kanonska formulacija

Dat je diskretni LIF neuron:

$$u_t = (3/4) v_t + (1/2) x_t$$

$$s_t = 1 \text{ ako je } u_t \geq 1, \text{ inače } 0$$

$$v_{t+1} = u_t - s_t$$

Početno stanje: $v_0 = 0$

Ulazni niz: $x = [1, 1, 1, 0, 1, 1, 0, 1]$

Traži se:

A-IZVOD: kratko objasniti dinamiku sistema i dokazati invarijantu $0 \leq v_t < 1$;

B-TABELA: dati tabelu za $t = 0..7$ sa x_t, v_t, u_t, s_t i v_{t+1} u razlomcima i decimalama;

C-KOD: napisati Python kod koji računa iste vrednosti i štampa RESULTS_JSON;

RESULTS_JSON: polja `u_fraction`, `u_decimal`, `s`, `v_fraction`, `v_decimal`, `spike_count`, `invariant_holds`.

A.3 GO-3 — Kanonska formulacija

Data je ulazna matrica X dimenzije 4x4:

$[[1, 2, 0, 1],$

$[3, 1, 2, 2],$

$[0, 1, 3, 1],$

$[2, 2, 1, 0]]$

Dat je kernel K dimenzije 3x3:

```
[[ 1, 0, -1],
 [ 2, 0, -2],
 [ 1, 0, -1]]
```

Traži se validna 2D matematička konvolucija sa stride=1 i bez padding-a.

Traži se:

A-RUCNI_REZULTAT: rotirati kernel za 180° i ručno izračunati sve izlazne elemente;

B-ANALIZA: objasniti izlazne dimenzije, broj množenja i sabiranja po izlaznom elementu i ukupno, te row-major indeksiranje;

C-KOD: napisati C funkciju conv2d_valid_3x3 i mali main primer;

RESULTS_JSON: output_h, output_w, manual_output, mul_per_output, add_per_output, mul_total, add_total.

A.4 GO-4 — Kanonska formulacija

Data je 4x4 matrica G sa simbolima {1,2,3,4}:

```
[[1,0,0,4],
 [0,4,0,0],
 [2,0,4,0],
 [0,3,0,1]]
```

Nula znači prazno polje. Zadato je da svaki red i svaka kolona moraju sadržati simbole 1..4 tačno jednom.

Traži se:

A-LOGIKA: dati početne skupove kandidata posle jednog prolaza eliminacije po redu i koloni, izbrojati prazna polja i singletone, i navesti najmanje četiri prisilna koraka;

B-BITMASKE: objasniti kodiranje kandidata u u8 masku, prepoznavanje singletona i zašto je za proveru jedinstvenosti dovoljno brojati do 2;

C-ZIG_KOD: napisati Zig rešavač sa funkcijama popcount4, singletonValue, candidatesMask i solveUnique;

RESULTS_JSON: empty_cells_initial, initial_singletons, unique, solution_count_capped, solution.

A.5 GO-5 — Kanonska formulacija

Dat je seed-vektor $V = [11, 7, 19, 4, 23, 8]$, deterministički Mealy transduktor sa stanjima $\{q_0, q_1, q_2, q_3\}$, ulaznim alfabetom $\{0, 1, 2\}$, izlaznim alfabetom $\{A, B, C\}$ i početnim stanjem q_0 . Parametri sistema generišu se pravilima $a_q = (V[q] + q) \bmod 4$ i $b_q = (V[q+2] + 2q) \bmod 3$, dok su sledeće stanje i izlaz definisani sa $\delta(q, x) = (q + a_q + x) \bmod 4$ i $\lambda(q, x) = (b_q + q + 2x) \bmod 3$.

Iz istog seed-vektora generišu se tri kandidatske reči W_0, W_1 i W_2 dužine 6 po pravilu $W_k[i] = (V[(i+k) \bmod 6] + i + 2k) \bmod 3$. Dat je i posmatrani izlaz $Y_{\text{obs}} = \text{"ACCB BB"}$ uz pretpostavku da je nastao od tačno jednog kandidata i tačno jedne simbolske greške u izlazu.

Traži se: A-GENERISANJE — izračunati parametre, tabelu transduktora i sve tri reči sa međukoracima; B-RUCNA_SIMULACIJA — ručno simulirati sva tri kandidata i izračunati putanje stanja, čiste izlaze i Hamming-ova rastojanja; C-DOKAZ_JEDINSTVENOSTI — identifikovati jedino validno objašnjenje, uz inverznu proveru i isključenje svih alternativa; D-ZIG_KOD — napisati Zig 0.16.x+ verifikator koji generiše instancu, simulira sve kandidate, traži sva validna objašnjenja sa tačno jednom greškom i izbacuje RESULTS_JSON; RESULTS_JSON — seed, parametre, reči, candidate_outputs, candidate_distances, pobednički kandidat, clean_output, state_path, error tuple i unique.

Aneks B. Preporučeni format prijave rezultata

Za svaku objavu ili interni izveštaj preporučuje se da se prijave dve tabele: 'all runs' i 'best by model'. U prvoj tabeli moraju biti sačuvani svi izvedeni runovi sa punim metapodacima. U drugoj se po modelu i testu prikazuje samo reprezentativni najbolji rezultat. Obe tabele su potrebne: prva zbog transparentnosti, druga zbog čitljivosti.

Minimalni skup kolona za tabelu rezultata je: Test, Model, Rig, tok/s, Total_s, Quality_10, Note. Po potrebi se dodaju TTFT, prompt tokeni, output tokeni, context length, quantization, template režim, flash attention status i CPU/GPU offload parametri.

Aneks C. Predlog kompozitnog tumačenja

Test	Preporučena težina u kompozitu	Glavni signal
GO-1	0.20	Formalna logika, minimalnost, jedinstvenost
GO-2	0.20	Egzaktna numerika, dokaz, Python verifikacija
GO-3	0.15	Ručni račun, C implementacija, memorijski model
GO-4	0.20	Integrisani test logike, bitmaski, Zig sintakse i JSON discipline
GO-5	0.25	Determinističko generisanje instance, simulaciona disciplina, jedinstvenost i Zig 0.16.x+ verifikacija

Tabela 8. Preporučeno tumačenje kompozitnog benchmark rezultata.

Aneks D. Operativni vodič za ocenjivanje GO-5

D.1 Provera A-sekcije. Evaluators prvo proverava da li su parametri a i b tačno izvedeni iz seed-vektora i da li su W0, W1 i W2 zaista rezultat zadate formule. Ako je A-sekcija pogrešna, odgovor se i dalje može čitati do kraja radi klasifikacije pada, ali ne može biti visoko ocenjen, jer model više ne rešava zadatak instancu nego sopstvenu pogrešnu varijantu problema.

D.2 Provera B-sekcije. Zatim se proverava da li su sva tri kandidata ručno simulirana bez preskakanja koraka i da li se putanje stanja, čisti izlazi i Hamming-ova rastojanja slažu sa tabelama. Ako je samo jedan kandidat tačno simuliran, a drugi ili treći nisu, to se tretira kao simulacioni drift, a ne kao sitna greška. GO-5 je namerno projektovan tako da sva tri kandidata moraju biti obrađena, jer upravo treći ili drugi kandidat često razotkrije gubitak pažnje.

D.3 Provera C-sekcije. Tvrdnja o jedinstvenosti ne ocenjuje se samostalno, već samo u odnosu na tačnost B-sekcije. Evaluators proverava da li model nije samo pronašao jedan kandidat na rastojanju 1, već i da li je eksplicitno isključio sve druge mogućnosti. Inverzna provera je obavezna zato što tera model da pređe iz čistog forward rezonovanja u kontrafaktualno ispitivanje susednih mogućnosti.

D.4 Provera D-sekcije. Zig kod se proverava na dve ravni: algoritamskoj i jezičkoj. Algoritamski, kod mora sam da generiše instancu, da simulira sve kandidate i da pronađe sva validna objašnjenja sa tačno jednom greškom. Jezički, kod mora biti kompatibilan sa Zig 0.16.x+ standardnom bibliotekom, bez zastarelog pre-0.16 I/O obrasca i bez lažno modernog, ali nekompatibilnog API stila. Kod koji razume problem, ali ne radi iz kutije, ne spada u vrhunsku klasu rezultata.

D.5 Provera RESULTS_JSON i završna ocena. JSON se proverava poslednji, ali mu se veruje najmanje: on mora biti rezultat prethodnih sekcija, a ne dekorativni sažetak. Ako candidate_outputs ili candidate_distances nisu saglasni sa ručnom simulacijom, evaluator ih tretira kao suštinsku grešku. Završna ocena se dodeljuje tek pošto su odgovor, kod i JSON međusobno usklađeni. Ako model pogodi konačni matching_word_index, ali je do njega došao preko pogrešne B-sekcije ili nekompatibilnog Zig-a, rezultat se klasifikuje kao delimično tačan, ali operativno nepouzdan.

Preporuka za praktičnu upotrebu je da se uz kanonsku javnu instancu GO-5 održava i mali skup audit-varijanti sa istim formalnim pravilima, ali drugim seed-vektorima i drugim posmatranim izlazima. Na taj način benchmark ostaje proverljiv i standardizovan, ali se smanjuje rizik da modeli vremenom nauče samo jedan javni primer umesto same metodologije rešavanja.